Querying RMBS trustee data in natural language

Utkarsh Malhotra · November 2026

The problem

Monthly trustee data for about 300 Australian RMBS deals sits in Oracle. Each deal holds about 2,000 loans going back 5 years, so the table runs to about 36 million loan-month records. Each record has arrears status, prepayments, LVR, interest rate, balance and a suburb that links to local house prices. The goal is to let people ask questions of this data in ordinary language and get answers they can rely on.

The main challenge is consistent definitions rather than data volume, since a well partitioned table of 36 million rows answers aggregate queries in seconds, as the prototype below confirms. A question such as "What are arrears on the 2021 vintage in Western Sydney?" could mean 30 or 90 days past due, by count or by balance, at the latest month or averaged over the year, with hardship loans included or excluded. A model writing SQL directly against trustee tables would choose one of those readings without stating it, and return a number that appears authoritative, which is a greater risk for a credit team than returning no answer.

The approach

The language model is used to interpret the question, while all calculation is left to the database. The model converts the question into a small structured query spec, which records the metric, how to group and filter it, and which months to cover. A compiler turns the spec into SQL using metric definitions held once in code, and the model then writes a short answer from the result table, with a check confirming that every number in that answer appears in the table. Where a question is ambiguous the system asks for clarification, and where the data cannot answer it the system declines and explains why.

Each answer comes with the spec, the Oracle SQL and the result table, so an analyst can verify it in about 10 seconds.

The design at production scale

1. Standardise the data before any model sees it. Trustee reports follow different conventions, in several ways that affect the numbers.

- One trustee reports arrears in days, another in missed payments.
- Hardship loans may or may not be counted in arrears.
- A clean-up call can be reported as an ordinary repayment.
- Corrected months are resubmitted on top of the originals.

The first step is therefore to bring every trustee's data into 1 standard table, where each loan keeps the same ID over time and the reason it left the pool is recorded properly, whether it was repaid early, bought back, defaulted or called. Current LVR is worked out once, by updating each property's value with local house prices, and saved alongside the loan. Without this step, a clean-up call would eventually appear as a wave of early repayments and produce an artificial spike in the prepayment rate.

2. Define each metric once. Every metric has a single definition held in code, so that 90+ arrears are measured by balance and exclude hardship, CPR counts voluntary prepayments only and is compounded over the period, and indexed LVR is weighted by balance.

The dimensions (deal, originator, vintage, state, suburb, LVR band, rate type, interest only) are listed with their allowed values, and because the model can only choose from these lists, it cannot introduce a metric of its own.

3. The model writes a spec, not SQL. The spec has a fixed shape, with 1 to 3 metrics, up to 2 groupings, filters, a date range and an optional ranking. Completing a short structured form is considerably more reliable than writing complex SQL from scratch, particularly over a pool that shrinks every month as loans repay, and the form is validated before any query runs.

4. Match names, then compile.

- People write deal and place names loosely, such as "Harbour 21-1", "blacktown" or "HARB21-1". Exact matches and known aliases resolve directly, close misspellings resolve by score, and a name that fits several values comes back as a question.
- The compiler then writes the query, and produces the same query in a version that Oracle can run, using only features Oracle supports.

5. Write the answer from the result, then check it. The answer text is written from the result table only, and every number in it is checked against the table. If any number fails the check, the user receives a plain summary of the table instead.

6. Test it on a fixed question set. A set of real questions with hand-checked answers runs after every change to the prompts, the model or a metric definition. The set includes ambiguous questions, to confirm that the system asks for clarification rather than guessing, and it provides the evidence that the system is accurate rather than merely plausible.

At 36 million rows, most questions only need totals by deal and month, so a small summary table, prepared in advance, answers them quickly, while the full loan-level detail remains available for more detailed analysis. In production, the system would only read from a copy of the database, never change it, with a time limit on every query and access controls for deals under NDA.

What the prototype shows

The prototype runs the full design on synthetic data built with the same reporting problems, covering 50 deals, 21,945 loans and 773,336 loan-months from 2021-10 to 2026-09, in 2 different trustee formats. Interest rates follow the RBA cash rate and house prices follow suburb indices, and 90+ arrears rise from 0.18% to a peak of 1.54% in 2024-08, then fall back to 0.47%.

The cleaning is tested row by row. The raw trustee files hold 774,057 rows, including 721 resubmitted duplicates, and one format reports its 7 clean-up calls as ordinary repayments. After cleaning, the canonical table matches the simulated truth on all 773,336 loan-months, with 0 differences.

The compiler is tested against hand-written SQL. 49 queries were written by hand and compared with the compiler's output, and all 49 returned the same answers, with the largest difference (1.6e-15) well below any practical significance. Every compiled query also runs in a form compatible with Oracle.

Data volume is not a constraint. With the data copied 36 times, to 27,840,096 loan-months, each of the 34 test questions with a numeric answer comes back in under 0.75 seconds on a single computer. Even when limited to 1 processor core, the slowest takes 3.34 seconds. These timings come from DuckDB, the database used in the prototype, and performance on Oracle would depend on table partitioning and indexing, which were not tested.

Compared with direct SQL. The same 40 questions were run through a second setup, in which a model writes DuckDB SQL directly against the same clean tables, with the same metric definitions written into its prompt. The 2 approaches performed more similarly than the design rationale above would suggest.

On the 34 questions with a numeric answer, direct SQL got 33 right and the compiled approach 31. Neither returned an incorrect number (0 for direct SQL, 0 for the compiled approach), and every miss on both sides was a refusal to answer.

- The compiled approach didn't recognise a state written out in full ("Queensland", stored as QLD).
- On the 2 questions phrased in missed payments, it asked which day threshold was meant instead of converting payments to days.
- On the 4 ambiguous questions, it asked a clarifying question each time, whereas direct SQL declined all 4 without explanation, which the scoring counts as incorrect.

Overall, the compiled approach got 37 of 40 (92.5%, 95% interval 80.1% to 97.4%) and direct SQL 35 of 40 (87.5%, interval 73.9% to 94.5%). The 2 confidence intervals overlap almost entirely.

The results indicate that, on clean data, the metric definitions account for most of the accuracy, whether a compiler applies them or a prompt carries them. Direct SQL performed well because its prompt contained the full set of definitions written out in prose, a copy that would need to be kept in line with the governing

definitions manually. The benefits of the compiled approach lie largely outside what this test scores, namely a single definition per metric used consistently, the spec and SQL behind every answer, a check on every number in the text, and a clarifying question in place of a silent refusal. Direct SQL was not tested against the raw trustee tables, where the traps described below would have the greatest effect.

category	questions	compiled approach	direct SQL
simple	10	9 (90%)	10 (100%)
grouped	10	10 (100%)	10 (100%)
time	6	6 (100%)	6 (100%)
trap	8	6 (75%)	7 (88%)
ambiguous	4	4 (100%)	0 (0%)
unsupported	2	2 (100%)	2 (100%)
total	40	37 (92%)	35 (88%)

The traps in the test set

The 8 trap questions target errors that produce valid SQL and wrong credit numbers, and on this data the effects are large.

- Counting clean-up calls as prepayments takes CST 2019-1's 2025 CPR from 21.6% to 100.0%.
- Counting the exit month as a live loan adds 597 loans that no longer exist to the November 2025 pool.
- A simple average instead of a balance-weighted one moves pool LVR from 51.96% to 50.92%.

With the definitions in its prompt, direct SQL avoided these too (7 of 8, against 6 for the compiled approach). These questions therefore measure what goes wrong without the definitions, rather than the additional value of the compiled approach.

A fourth mistake, averaging the monthly prepayment rates instead of compounding them, was considered and excluded, because at the pool's prepayment rate of 18.1% the correct and incorrect methods differ by only 1.2e-05, too little to distinguish a careful answer from a careless one.

Next steps and limitations

The next tools to build go beyond aggregation, starting with roll rate matrices, prepayment curves by seasoning and cohort loss curves, then arrears sensitivity to house price falls.

The house price analysis requires particular care, because between 2022 and 2024 rate rises pushed arrears up and house prices down in the same suburbs at the same time, and a simple regression would attribute to house prices what was largely a rate effect. The tool should therefore control for payment shock and state clearly which effects it cannot separate.

The test set should also expand through real use, with every question logged and every failure added, and the first improvement it identifies is support for state names written in full.

The work has 3 main limitations.

- The data is synthetic.
- The test set consists of 40 questions run once, so each category carries a wide confidence interval, and because current Claude models do not accept a sampling temperature, a rerun could produce different answers, a variation that has not been measured.
- The Oracle SQL is generated and syntax-checked, but has not been run on an Oracle database.